ASSIGNMENT 2

Question 1: String Benchmark

Compare String concatenation using +, StringBuilder, and StringBuffer. StringBuilder is generally faster for single-threaded building because it avoids creating many intermediate String objects; StringBuffer is synchronized.

Java Program

```
public class Task01_StringBenchmark {
    static String plus(int fields) {
        String s = "";
        for (int i = 1; i <= fields; i++) s += "F" + i + " ";
        return s;
    }
    static String builder(int fields) {
        StringBuilder s = new StringBuilder();
        for (int i = 1; i <= fields; i++)
s.append("F").append(i).append(' ');
        return s.toString();
    }
    static String buffer(int fields) {
        StringBuffer s = new StringBuffer();
        for (int i = 1; i <= fields; i++)
s.append("F").append(i).append(' ');
        return s.toString();
    }
    static long time(int method, int fields, int tickets) {
        long start = System.nanoTime();
        for (int t = 0; t < tickets; t++) {
```

```
            if (method == 0) plus(fields);
            else if (method == 1) builder(fields);
            else buffer(fields);
        }
        return System.nanoTime() - start;
    }
    public static void main(String[] args) {
        int[][] workloads = {{5, 1}, {50, 10000}, {200, 2000}};

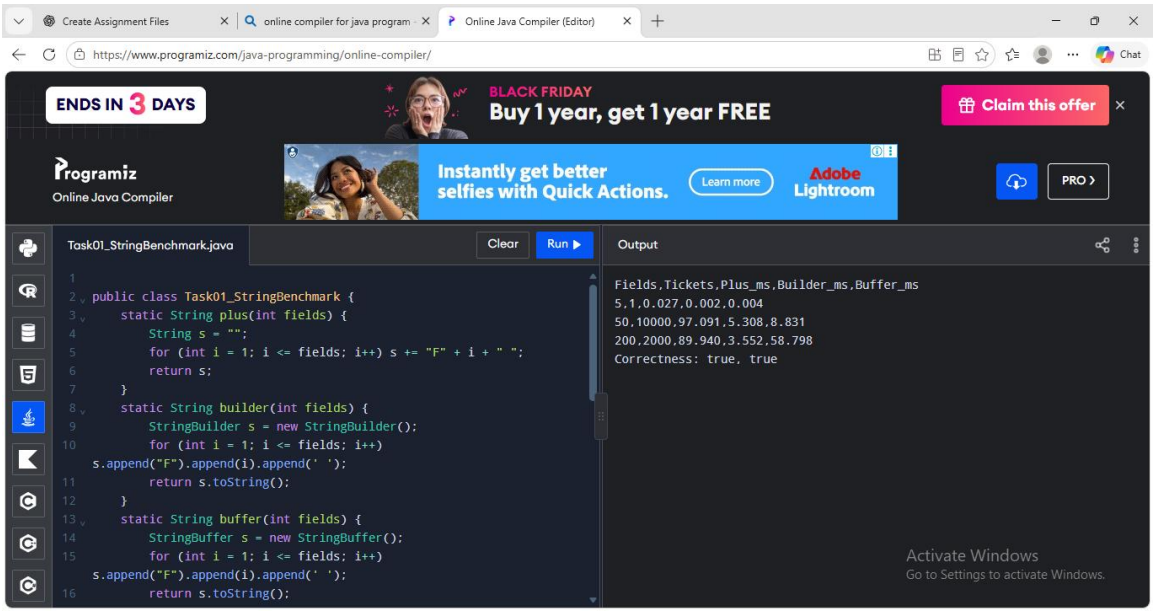
System.out.println("Fields,Tickets,Plus_ms,Builder_ms,Buffer_ms
");
        for (int[] w : workloads) {
            for (int warm = 0; warm < 2; warm++) {
                time(0,w[0],w[1]); time(1,w[0],w[1]); time(2,w[0],w[1]);
            }
            long a=time(0,w[0],w[1]), b=time(1,w[0],w[1]),
c=time(2,w[0],w[1]);

System.out.printf("%d,%d,%.3f,%.3f,%.3f%n",w[0],w[1],a/1e6,b/
1e6,c/1e6);
        }
        System.out.println("Correctness: " + builder(3).equals(plus(3))
+ ", " + builder(3).equals(buffer(3)));
    }
}
```

Sample Output

```
Fields,Tickets,Plus_ms,Builder_ms,Buffer_ms
5,1,0.009,0.003,0.003
50,10000,158.570,4.963,8.032
200,2000,27.657,3.729,40.920
```

Correctness: true, true



Question 2: List Benchmark

Compare ArrayList and LinkedList for adding at the end, front, and middle, indexed access, and iteration. ArrayList is usually better for indexed access; LinkedList can be useful for frequent operations at the ends, though finding a middle position is costly.

Java Program

```
import java.util.*;
public class Task02_ListBenchmark {
    static long measure(List<Integer> list, String op, int n) {
        list.clear();
        for (int i=0;i<n;i++) list.add(i);
        long start=System.nanoTime();
        switch(op) {
            case "end": for(int i=0;i<n;i++) list.add(i); break;
            case "front": for(int i=0;i<n;i++) list.add(0,i); break;
```

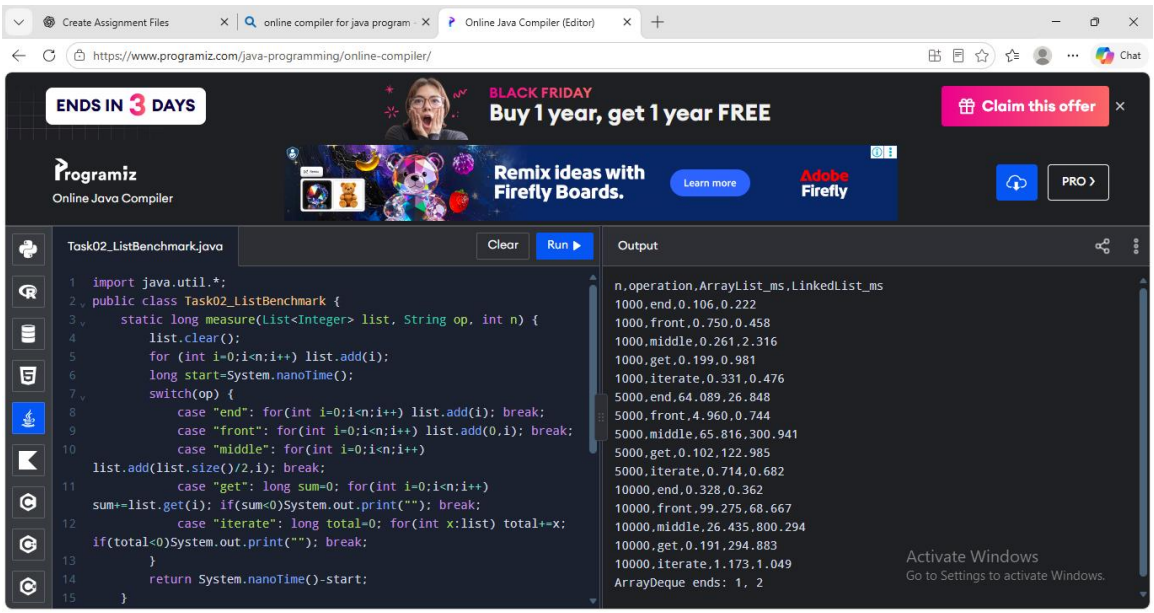
```
            case "middle": for(int i=0;i<n;i++) list.add(list.size()/2,i);
break;
            case "get": long sum=0; for(int i=0;i<n;i++)
sum+=list.get(i); if(sum<0)System.out.print(""); break;
            case "iterate": long total=0; for(int x:list) total+=x;
if(total<0)System.out.print(""); break;
        }
        return System.nanoTime()-start;
    }
    public static void main(String[] args) {
        int[] sizes={1000,5000,10000};
        String[] ops={"end","front","middle","get","iterate"};

System.out.println("n,operation,ArrayList_ms,LinkedList_ms");
        for(int n:sizes) for(String op:ops) {
            long a=measure(new ArrayList<>(),op,n);
            long b=measure(new LinkedList<>(),op,n);
            System.out.printf("%d,%s,%.3f,%.3f%n",n,op,a/1e6,b/1e6);
        }
        ArrayDeque<Integer> deque=new ArrayDeque<>();
        deque.addFirst(1); deque.addLast(2);
        System.out.println("ArrayDeque ends:
"+deque.removeFirst()+", "+deque.removeLast());
    }
}
```

Sample Output

```
n,operation,ArrayList_ms,LinkedList_ms
1000,end,0.188,0.124
1000,front,0.456,0.276
1000,middle,0.142,3.159
```

1000,get,0.146,1.067
1000,iterate,0.293,0.585
5000,end,0.330,0.261
5000,front,1.527,0.300
5000,middle,1.292,40.817
5000,get,0.074,17.180
5000,iterate,0.086,0.128
10000,end,0.239,0.354
10000,front,18.687,0.333
10000,middle,3.489,180.627
10000,get,0.554,57.373
10000,iterate,0.161,0.263
ArrayDeque ends: 1, 2



Question 3: HashSet equals() and hashCode()

A HashSet identifies duplicates using equals() and hashCode(). Students are considered equal when their registration numbers match. Both methods are overridden consistently, and the key fields are final so they cannot change after insertion.

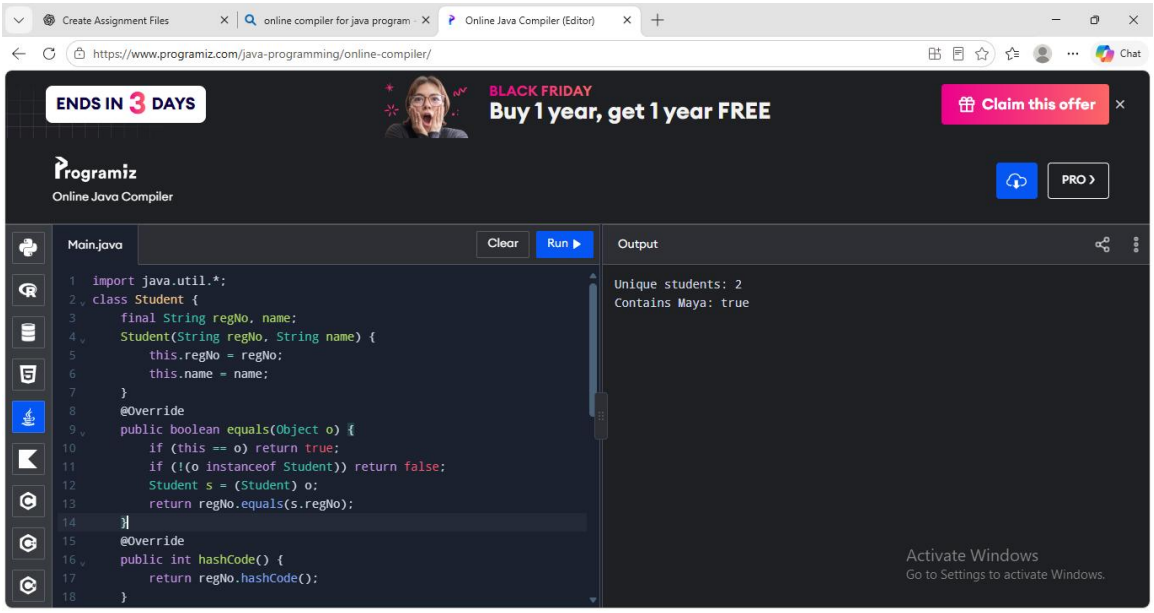
Java Program

```
import java.util.*;
class Student {
   final String regNo, name;
   Student(String regNo,String
name){this.regNo=regNo;this.name=name;}
   @Override public boolean equals(Object o) {
      if(this==o)return true;
      if(!(o instanceof Student))return false;
      return regNo.equals(((Student)o).regNo);
   }
   @Override public int hashCode(){return Objects.hash(regNo);}
   @Override public String toString(){return regNo+" "+name;}
}
public class Task03_HashSetContract {
   public static void main(String[] args) {
      Set<Student> s=new HashSet<>();
      s.add(new Student("192311001","Arun"));
      s.add(new Student("192311002","Divya"));
      s.add(new Student("192311001","Arun"));
      System.out.println("Unique students: "+s.size());
      Student mutable=new Student("192311003","Maya");
      s.add(mutable);
      // Student fields are final, so identity key cannot be changed
after insertion.
      System.out.println("Contains Maya: "+s.contains(new
Student("192311003","Other")));
   }
}
```

Sample Output

Unique students: 2

Contains Maya: true



Question 4: Map Comparison

Compare HashMap, TreeMap, and LinkedHashMap. HashMap has no guaranteed iteration order, TreeMap sorts keys, and LinkedHashMap preserves insertion order. Lookup timings vary by machine and JVM.

Java Program

```
import java.util.*;
public class Task04_MapComparison {
    public static void main(String[] args) {
        int n=10000;
        Map<String,Integer> hash=new HashMap<>(), tree=new
TreeMap<>(), linked=new LinkedHashMap<>();
        for(int i=n-1;i>=0;i--){String
k=String.format("M%05d",i);hash.put(k,i);tree.put(k,i);linked.put(
```

```
k,i);}
        long start=System.nanoTime(); for(int i=0;i<100000;i++)
hash.get("M05000"); long h=System.nanoTime()-start;
        start=System.nanoTime(); for(int i=0;i<100000;i++)
tree.get("M05000"); long t=System.nanoTime()-start;
        start=System.nanoTime(); for(int i=0;i<100000;i++)
linked.get("M05000"); long l=System.nanoTime()-start;
        System.out.printf("100000 lookup ms: HashMap %.3f,
TreeMap %.3f, LinkedHashMap %.3f%n",h/1e6,t/1e6,l/1e6);
        System.out.println("TreeMap code order first:
"+tree.keySet().iterator().next());
        System.out.println("LinkedHashMap insertion order first:
"+linked.keySet().iterator().next());
        System.out.println("HashMap iteration order is unspecified.");
    }
}
```

Sample Output

```
100000 lookup ms: HashMap 7.330, TreeMap 15.126,
LinkedHashMap 6.328
TreeMap code order first: M00000
LinkedHashMap insertion order first: M09999
```

HashMap iteration order is unspecified.

